Verified Projection-Event Calculus in Lean 4: Bundled Arbitrary-Partition Pinching, Lüders Retractions, and CHSH Interoperability

Bernhard Mueller*
Pragma Research Inc., Washington, United States

July 23, 2026

Paper release: r1577 **Released:** July 23, 2026

Abstract

Finite quantum measurement looks simple on paper: multiply matrices, take a trace, and normalize. In Lean, each step crosses a different interface and every side condition must be supplied. We build a Lean 4 library for projection events over finite complex matrix algebras. A projective partition defines a star-subalgebra of matrices that commute with its projectors and a complex-linear pinching map onto that subalgebra. The map is unital, positive, trace preserving, and idempotent; its range and fixed points are exactly the partition commutant, and it satisfies the bimodule law, Hilbert–Schmidt Pythagorean and contraction identities, and a trace-duality uniqueness theorem. A second expectation averages onto the commutative span of the partition. The two maps satisfy tower laws, the average preserves the Born statistics of the partition, and conditioning on a partition member commutes with averaging. Lüders conditioning is available as a total matrix formula and as a typed state update; the fixed states are exactly the states assigning weight one to the event. A checked bound dominates every state expectation by the operator norm of the observable, which turns an operator-norm Tsirelson theorem, interoperable with Mathlib's Clauser-Horne-Shimony-Holt (CHSH) interface, into a state-level CHSH bound for projection events. We compare the library with Mathlib, Physlib, Lean-Quantum, Lean-QIT, and Isabelle/HOL, and we record the Mathlib gaps met during construction together with the workarounds that closed them. The artifact pins its Lean and Mathlib revisions and records declaration-level axiom dependencies. Complete positivity, a Physlib correspondence, rank-one classicality, and a CHSH sharpness witness are outside the formalization.

Keywords: formal verification, Lean 4, projective measurement, partition pinching, Lüders update, CHSH

Mathematics Subject Classification (2020): Primary 68V20; Secondary 68V15, 81P15, 81P40, 46L53.

*Corresponding author: bernhard@floatingpragma.ai

1 Introduction

Finite quantum measurement is short on paper. A proof calls a matrix a state, inserts a projection, normalizes the result, and moves on. Lean asks for every condition skipped in that sentence. The matrix must be positive and have unit trace. The projection must be Hermitian and idempotent. Normalization needs a nonzero denominator. Later norm estimates require a further set of instances.

Machine-checked quantum information is an active area, spread over several systems and several design philosophies. Mathlib [19], the central mathematics library of Lean 4 [6], contains an order-form CHSH/Tsirelson theorem and the `IsCHSHTuple` interface [13]. Physlib's `QuantumInfo.Finite.Pinching` constructs the spectral pinching of a state as a completely positive trace-preserving map and proves commutation, fixed-point, Pythagorean, and pinching-bound results [16, 17]. Lean-Quantum provides basis-independent infrastructure for states, channels, tensor products, Choi and Kraus representations, and trace inequalities [11]; Lean-QIT develops a compositional infrastructure for quantum-information theory [23]. Zhao and Yu formalize CHSH rigidity in Lean 4 [22]. Isabelle/HOL developments cover projective measurement, quantum computation, and the CHSH upper bound [2, 7–9]. These developments choose different carriers for states and different bundling conventions, and results proved in one seldom apply verbatim in another. The engineering question underneath is interface design: which objects stay bare predicates, which get bundled, and how finished results are handed to the interfaces a large library expects. We make no priority claim for projective measurement, pinching, or Tsirelson's inequality.

The library described here packages the side conditions of finite projective measurement into a projection-event API over Mathlib matrices. Its main input is an arbitrary projective partition of the identity. The partition induces two canonical expectations: the pinching, which deletes off-block terms and maps the matrix algebra onto the partition commutant, and the average, which projects further onto the commutative span of the partition. The same event definitions feed the Lüders update and the CHSH adapter. This keeps algebraic identities separate from results that use trace, positivity, or state normalization.

The mathematics is classical: Lüders introduced the selective update rule [3, 12]; pinching by a family of orthogonal projections is a standard averaging operation in matrix analysis [1] and in quantum information [10]; the bimodule and trace-duality laws proved here are the finite-matrix forms of the defining properties of conditional expectations onto subalgebras [18, 20]; and the CHSH combination with its quantum bound goes back to Clauser, Horne, Shimony, and Holt [5] and to Tsirelson [4]. The contribution of the library is the checked interface: definitions bundled the way Mathlib expects, exact range and uniqueness theorems for both expectations, and adapters into existing Mathlib results.

Lean declaration identifiers are printed throughout in monospaced type, for example `partitionPinching_idem`; the underscore is part of the identifier. To locate a declaration, search for the exact printed identifier, after removing only TeX's protective backslash before each underscore, in the `EventAlgebra/*.lean` files. In the public repository these files are under `Lean/EventAlgebra/`; in the standalone artifact they are under `event-algebra-lean/EventAlgebra/`. Table 1 maps each module name to its subject matter, and the top-level `EventAlgebra.lean` file imports all seven modules.

The submitted code adds seven pieces:

(i) an arbitrary supplied projective partition, independent of a selected state or spectral resolu-

tion, with its commutant bundled as a `StarSubalgebra` and its span bundled with multiplication, star, commutativity, and centrality theorems;

(ii) partition pinching bundled as a complex `LinearMap`, with exact range and fixed-point theorems, positivity, unitality, trace preservation, the commutant bimodule law, Hilbert–Schmidt geometry, and map-level uniqueness;

(iii) partition averaging bundled as a complex `LinearMap` onto the span, with exact range, trace duality, map-level uniqueness, tower laws against the pinching, preservation of partition Born statistics, and a classical-conditioning collapse theorem;

(iv) a typed nonzero-weight Lüders state update, a support theorem and an unguarded fixed-point characterization that exhibit conditioning as a retraction onto its certainty set, and naturality of the typed update under partition pinching for commutant events;

(v) an event-to-observable CHSH client that discharges Mathlib-compatible selfadjointness, involutivity, and cross-commutation hypotheses before applying a complementary norm-form bound;

(vi) a checked state-expectation bound that converts the norm-form theorem into a state-level CHSH bound; and

(vii) a reproducible, version-pinned artifact with per-declaration axiom output, no proof placeholders, and a documented catalog of the Mathlib gaps, scoped-instance hazards, and workarounds met during construction (Section 9).

Four results are outside the current library. Neither expectation is bundled as a completely positive trace-preserving channel. No theorem identifies an arbitrary partition with Physlib's spectral partition. There is no rank-one classical-representation theorem. The CHSH bounds carry no sharpness witness: nothing asserts that $2\sqrt{2}$ is attained.

Section 2 fixes the mathematical and Lean interfaces. Sections 3 and 4 treat projection events and selective update. Section 5 gives the bundled partition pinching and its geometry, and Section 6 the averaging expectation and the two-level structure. Sections 7 and 8 describe the state functional and the CHSH clients. Section 9 records the Mathlib engagement. Section 10 reports the audit methodology, and Section 11 positions the development feature by feature.

2 Mathematical and formal setting

Let $\mathrm{M}_n(\mathbb{C})$ be the complex $n \times n$ matrices, with conjugate transpose $X \mapsto X^*$, trace Tr, and identity $\mathbf{1}$. Following the standard finite-dimensional setting [21], a projection event is a Hermitian idempotent $P = P^* = P^2$, a state is a positive-semidefinite matrix ρ with $\mathrm{Tr}(\rho) = 1$, and the trace pairing is $\mu_\rho(M) = \mathrm{Tr}(\rho M)$.

Lean represents these interfaces as predicates over Mathlib matrices: `IsEvent P` is the conjunction $P^* = P$ and $P^2 = P$; `IsState rho` is positive semidefiniteness and unit trace. For functions that return states, the library defines the subtypes `ProjectionEvent n` and `StateMatrix n`. The split is practical. Algebraic lemmas use raw matrices. A function advertised as a state update returns a value whose type carries the state conditions.

Table 1 lists the seven source modules. The comparison in this work concerns their compiled interfaces and uses, not the number of declarations in each file.

Module	Principal interface
`Basic`	projection events, states, trace pairing, closure and Born weight bounds
`Lueders`	totalized matrix formula, typed state update, repeatability, composition, support, and fixed points
`Partition`	projective partitions, bundled commutant, bundled linear
`Pinching`	pinching, exact range, geometry, bimodule law, and naturality
`PartitionAverage`	partition span with closure, commutativity, and centrality; bundled averaging expectation with exact range, uniqueness, tower laws, statistics preservation, and classical conditioning
`StateExpectation`	trace pairing bundled as a complex-linear functional, normalization and positivity
`Tsirelson`	CHSH ring identity, norm theorem, Mathlib interoperability, and event-level matrix client
`ExpectationBound`	state-expectation operator-norm bound and the state-level CHSH corollary

Table 1: Module structure of the checked development.

Three Mathlib scopes affect compilation. `ComplexOrder` equips $\mathbb{C}$ with the partial order in which $0 \leq z$ means that z is real and nonnegative. `MatrixOrder` supplies the Loewner order. `Matrix.Norms.L2Operator` activates the finite-matrix operator norm and C*-ring instances used by the matrix CHSH theorems. Without the right scope, Lean may report an instance problem even though the required theorem is in scope; Section 9 describes the failure modes.

Documentation tags classify results as *algebra-only* or *trace-dependent*. These tags do not define an import hierarchy. They record whether a proof uses ring, star, and norm structure alone, or also uses $\mathrm{Tr}(\rho M)$, positivity, or normalization.

3 Projection events and the trace pairing

The event layer proves the expected closure rules. Zero, identity, and $\mathbf{1}-P$ are events. Orthogonality is symmetric. Orthogonal sums and products of commuting events are events. Subevent absorption holds on both sides, and every event is positive semidefinite. The declaration `IsEvent.one_sub_two_smul_involution` proves that $\mathbf{1}-2P$ is a selfadjoint involution. The CHSH adapter in Section 8 uses this result.

The trace pairing is additive, homogeneous, compatible with finite sums, and obeys the sandwich identity

$$\mathrm{Tr}(P\rho P) = \mathrm{Tr}(\rho P)$$

for idempotent P. Hermiticity of ρ and P implies that the pairing is real. Positivity of ρ and the event property imply $0 \leq \mu_\rho(P)$ in Mathlib's order on $\mathbb{C}$; for a state the upper bound is $\mu_\rho(P) \leq 1$. The order-form statements are strictly stronger than their real-part corollaries, and both forms are provided.

The declaration `isEvent_add_and_bornWeight_add_of_orthogonal` returns two facts: the orthogonal sum is an event, and its Born weight is the sum of the two Born weights. The event and orthogonality hypotheses carry the first conjunct; the weight identity is linear and uses none of them.

4 Lüders update as a partial state interface

For raw matrices, the library uses the totalized formula

$$\mathcal{L}_P(\rho) = \mu_\rho(P)^{-1} P\rho P.$$

Lean's field inverse maps zero to zero, so the formula is total and covers algebraic identities in the zero-weight case. Its return type is a raw matrix and carries no state guarantee. The typed operation is

$$\texttt{luedersStateUpdate}: \ \texttt{StateMatrix n} \to \texttt{ProjectionEvent n} \to (\mu_\rho(P) \neq 0) \to \texttt{StateMatrix n}.$$

This operation assumes `NeZero n`. The unique 0×0 matrix cannot have unit trace, so a state-returning function must exclude that dimension.

Proposition 1 (Checked Lüders interface)**.** *For a state ρ, event P, and nonzero outcome weight:*

(i) the raw formula is positive semidefinite and has trace one (`luedersUpdate_isState`);

(ii) the updated state assigns weight one to P (`bornWeight_luedersUpdate_self`);

(iii) update is an idempotent retraction (`luedersUpdate_idem`);

(iv) updates by commuting events compose and commute (`luedersUpdate_luedersUpdate_of_commute` and `luedersUpdate_comm`);

(v) when the state commutes with the event, the update is the normalized block restriction $\mu_\rho(P)^{-1}\rho P$ (`luedersUpdate_of_commute`); and

(vi) the fixed-state equivalence

$$\mathcal{L}_P(\rho) = \rho \quad \Longleftrightarrow \quad \mu_\rho(P) = 1$$

holds without a separate nonzero-weight hypothesis (`luedersUpdate_eq_self_iff`).

The fixed-point theorem needs no nonzero-weight guard. A state fixed by a zero-weight totalized update would equal the zero matrix, which has the wrong trace. The raw function covers the zero-weight case; the state theorem does not carry a redundant premise.

The module assembles these results into a fixed-point description of conditioning. The set `certainStates` collects the states that assign weight one to P. Conditioning any state of nonzero weight lands in that set in a single step (`luedersUpdate_mem_certainStates`), acts on the set as the identity, and, among states, fixes exactly its elements. Conditioning on P is therefore a retraction of the nonzero-weight states onto its own fixed-point set. The identity case rests on a support theorem, `mul_eq_self_of_bornWeight_one`: a state certain of P absorbs the event on both sides, $\sigma P = P\sigma = \sigma$. The checked proof compresses σ by the complement event, whose weight vanishes; positive semidefiniteness forces the zero-trace compression to vanish; and the Cauchy–Schwarz property of the state's quadratic form eliminates $\sigma(\mathbf{1} - P)$ itself.

5 Bundled arbitrary-partition pinching

A `ProjectivePartition n k` consists of projections $(P_i)_{i<k}$, pairwise orthogonality, and $\sum_i P_i = \mathbf{1}$. It induces

$$\mathcal{E}_\pi(X) = \sum_i P_i X P_i, \qquad N_\pi = \{X : XP_i = P_i X \text{ for every } i\}.$$

The code defines N_π as `ProjectivePartition.commutant`, a `StarSubalgebra` defined using Mathlib's centralizer, and bundles $\mathcal{E}_\pi$ as `partitionPinchingLinearMap`. The membership theorem `mem_commutant_iff` reduces membership to the pointwise commutation equation.

The range is a block-diagonal commutant and need not be commutative. For this reason, we do not call it a "classical algebra"; the classical layer is the span of Section 6. The library has no theorem identifying the span with the center of the commutant, and no rank-one classical-representation theorem.

Theorem 1 (Bundled retraction and exact range)**.** *For every projective partition π, the map $\mathcal{E}_\pi$ is complex linear, unital, positive, trace preserving, and idempotent. It lands in N_π and fixes N_π pointwise. Consequently*

$$\mathcal{E}_\pi(X) = X \iff X \in N_\pi, \qquad \operatorname{range}(\mathcal{E}_\pi) = N_\pi$$

where the second equality is the equality of the linear-map range and the underlying submodule of the bundled commutant.

The corresponding declarations are `partitionPinching_unital`, `partitionPinching_posSemidef`, `trace_partitionPinching`, `partitionPinching_idem`, `partitionPinching_eq_self_iff_mem_commutant`, and `range_partitionPinchingLinearMap`. State preservation is exposed by `partitionPinching_isState`.

Theorem 2 (Bimodule and Hilbert–Schmidt structure)**.** *If $A, B \in N_\pi$, then*

$$\mathcal{E}_\pi(AXB) = A\mathcal{E}_\pi(X)B.$$

Pinching is also selfadjoint for the trace pairing and satisfies the Pythagorean identity

$$\operatorname{Tr}(X^*X) = \operatorname{Tr}(\mathcal{E}_\pi(X)^*\mathcal{E}_\pi(X)) + \operatorname{Tr}((X - \mathcal{E}_\pi(X))^*(X - \mathcal{E}_\pi(X))),$$

and hence contracts the squared Hilbert–Schmidt quantity. A commutant-valued complex-linear map with the same trace pairings against every commutant element equals `partitionPinching LinearMap`.

The corresponding declarations are `partitionPinching_bimodule`, `trace_conjTranspose_partitionPinching_mul`, `trace_partitionPinching_pythagoras`, `trace_partitionPinching_contraction`, `trace_partitionPinching_mul_commutant`, and `partitionPinchingLinearMap_unique`. Together they make the map a positive, unital, trace-preserving linear projection with a bimodule law and a trace-duality characterization; these are the identities that characterize the trace-preserving conditional expectation onto a subalgebra in the operator-algebra literature [18, 20]. The library proves the identities directly for this map. It does not instantiate an operator-algebra conditional-expectation or channel interface because complete positivity is not proved.

5.1 Lüders naturality

If $P \in N_\pi$, compression commutes with pinching. After normalization, trace preservation against commutant elements gives

$$\mathcal{E}_\pi(\mathcal{L}_P(\rho)) = \mathcal{L}_P(\mathcal{E}_\pi(\rho)).$$

The raw formula is `partitionPinching_luedersUpdate`. The theorem `partitionPinching_luedersStateUpdate` states the same naturality for `StateMatrix n` and `ProjectionEvent n`: the input and output are states, and the nonzero-weight proof is transported by `bornWeight_partitionPinching`. This checks the same operation through the bundled state and event interfaces.

6 Averaging on the partition span

The same partition carries a second canonical expectation. The *partition span*

$$D_\pi = \operatorname{span}\{P_i\}$$

is bundled as `ProjectivePartition.span`. Its closure package is proved by span induction.

Proposition 2 (The commutative layer)**.** *D_π contains every projector and the identity, and is closed under multiplication and conjugate transposition (`proj_mem_span`, `one_mem_span`, `mul_mem_span`, `conjTranspose_mem_span`). It is commutative (`span_mul_comm`). It is contained in the commutant (`span_le_commutant`), and every element of D_π commutes with every element of N_π (`mul_comm_of_mem_span_of_mem_commutant`): the span lies in the center of the commutant.*

Only the inclusion into the center is proved; the library does not claim the reverse inclusion, which fails in the presence of zero projectors. The averaging map is

$$\mathcal{A}_\pi(X) = \sum_i \frac{\operatorname{Tr}(XP_i)}{\operatorname{Tr}(P_i)}\, P_i,$$

bundled as `partitionAverageLinearMap`. Lean's zero-totalized inverse makes the terms of zero projectors vanish, so no nonzero-block hypothesis appears anywhere in the module: a zero projector contributes zero to both sides of every identity below.

Theorem 3 (Averaging expectation and exact range)**.** *For every projective partition π, the map $\mathcal{A}_\pi$ is complex linear, unital, positive, trace preserving, and idempotent. It lands in D_π, fixes D_π pointwise, and*

$$\mathcal{A}_\pi(X) = X \iff X \in D_\pi, \qquad \operatorname{range}(\mathcal{A}_\pi) = D_\pi.$$

Against every $C \in D_\pi$ the average is invisible to the trace pairing, $\operatorname{Tr}(\mathcal{A}_\pi(X)\,C) = \operatorname{Tr}(XC)$, and any D_π-valued complex-linear map with this property equals the bundled average.

The corresponding declarations are `partitionAverage_unital`, `partitionAverage_posSemidef`, `trace_partitionAverage`, `partitionAverage_idem`, `partitionAverage_eq_self_iff_mem_span`, `range_partitionAverageLinearMap`, `trace_partitionAverage_mul_of_mem`, and `partitionAverageLinearMap_unique`; state preservation is `partitionAverage_isState`. The uniqueness proof mirrors the pinching case: the difference lies in the star-closed span, is trace-orthogonal to its own conjugate transpose, and vanishes by faithfulness of the trace.

Theorem 4 (Two-level structure)**.** *The two expectations compose as a tower,*

$$\mathcal{A}_\pi \circ \mathcal{E}_\pi = \mathcal{A}_\pi = \mathcal{E}_\pi \circ \mathcal{A}_\pi,$$

(`partitionAverage_partitionPinching`, `partitionPinching_partitionAverage`). The average preserves the Born statistics of the partition, $\mu_{\mathcal{A}_\pi(\rho)}(P_j) = \mu_\rho(P_j)$ for every j (`bornWeight_partitionAverage`). For a partition member P_j of nonzero weight, averaging commutes with Lüders conditioning, and both composites collapse to the normalized projector:

$$\mathcal{A}_\pi(\mathcal{L}_{P_j}(\rho)) = \mathcal{L}_{P_j}(\mathcal{A}_\pi(\rho)) = \mathrm{Tr}(P_j)^{-1}\, P_j$$

(`partitionAverage_luedersUpdate`, `partitionAverage_luedersUpdate_proj`, `luedersUpdate_partitionAverage_proj`).

The collapse identity is the finite-matrix form of classical conditioning: on the commutative layer, conditioning on an observed partition member replaces the averaged state by the indicator of that member, normalized. The identity is stated for raw matrices; the only hypothesis is the nonzero outcome weight.

7 The bundled state expectation

For a matrix ρ, `stateExpectationLinearMap rho` bundles $M \mapsto \mathrm{Tr}(\rho M)$ as a complex-linear functional. If ρ is a state it maps identity to one. If ρ and M are positive semidefinite, then the expectation is nonnegative. The proof of `expectation_nonneg` uses a finite spectral decomposition of M, cycles the trace, and reduces the result to a sum of products of nonnegative diagonal entries; Section 9 explains why this elementary route was chosen. Additivity and homogeneity come from the bundled `LinearMap` interface and are not repeated as pointwise lemmas.

8 CHSH interoperability as a client

For selfadjoint involutions a_0, a_1, b_0, b_1 with cross commutation, set

$$S = a_0 b_0 + a_0 b_1 + a_1 b_0 - a_1 b_1.$$

The development first proves, in a bare ring,

$$S^2 = 4\mathbf{1} - (a_0 a_1 - a_1 a_0)(b_0 b_1 - b_1 b_0)$$

(`chsh_mul_self`). In a nontrivial unital C*-ring this yields the operator-norm bound $\|S\| \le 2\sqrt{2}$ (`tsirelson_bound`), the norm form of Tsirelson's inequality [4] for the CHSH combination [5]. The analytic half is short: selfadjoint involutions have norm one by the C*-identity (`norm_eq_one_of_selfAdjoint_involution`), commutators of norm-one elements have norm at most two (`norm_commutator_le_two`), and the square identity converts these bounds into $\|S\|^2 \le 8$. The precise Mathlib structure is a `NormedRing`, `StarRing`, `CStarRing`, and `Nontrivial`. The result is stated for a unital C*-ring.

The theorem `tsirelson_bound_of_isCHSHTuple` accepts Mathlib's `IsCHSHTuple`. The matrix instantiation, `matrix_tsirelson_bound`, activates Mathlib's scoped operator norm. The declaration `matrix_tsirelson_bound_of_events` accepts four projection events plus the four cross-commutation equations, constructs the dichotomic observables $\mathbf{1}-2P$, and returns the norm bound. This gives the event layer a compiled path into the CHSH theorem.

Mathlib's `tsirelson_inequality` is an order inequality in a star-ordered real algebra [13]. The theorems in this artifact are an operator-norm bound and its state-level corollary below; they provide no equality witness.

8.1 The state-level corollary

The bridge from norms to states is the checked bound

$$|\mathrm{Tr}(\rho M)| \le \|M\|$$

for every state ρ and observable M (`norm_expectation_le_l2_opNorm`). The proof diagonalizes the state, cycles the trace to $\mathrm{Tr}(D\,(V^*MV))$, dominates every diagonal entry of the conjugated observable by the operator norm (`norm_diag_entry_le_l2_opNorm`, by testing against a standard basis vector), uses that unitary conjugation does not increase the norm (`norm_eq_one_of_mem_unitaryGroup` with submultiplicativity), and sums the nonnegative eigenvalues to one. Composing with the event-level norm bound gives the state-level CHSH corollary (`matrix_state_tsirelson_bound_of_events`): for a state and four projection events with cross-party commutation,

$$\left|\mathrm{Tr}(\rho S)\right| \le 2\sqrt{2}$$

for the CHSH combination S of the dichotomic observables $\mathbf{1}-2P$. The pairing $\mathrm{Tr}(\rho S)$ is real by `star_bornWeight`, since S is selfadjoint, so the modulus bound is a two-sided real bound. No sharpness witness is claimed.

9 Working over Mathlib: scopes, gaps, and affordances

This section records what the construction consumed from Mathlib, where the library resisted, and which workarounds closed the gaps. The artifact repeats the catalog in the notes file `MATHLIB_NOTES.md`.

9.1 Scoped instances

The three scopes of Section 2 share a failure mode. The partial order on $\mathbb{C}$ and the Loewner order are scoped instances; every $0 \le z$ goal and every positive-semidefiniteness statement over $\mathbb{C}$ elaborates only when the scope is open, and a missing scope surfaces as an instance-resolution error far from its cause. `ComplexOrder` also carries the scoped ordered-ring structure on $\mathbb{C}$ that supplies multiplicativity of nonnegativity inside the complex order. There is no global norm on Mathlib matrices; the operator norm, the `NormedRing` structure, and the `CStarRing` instance arrive only under `Matrix.Norms.L2Operator`, so the entire Tsirelson section lives behind that scope. The `Nontrivial` instance for $n \times n$ complex matrices with $n \neq 0$ did not synthesize and is constructed by hand, entrywise. The matrix unitary group is a submonoid distinct from Mathlib's `unitary` bundle, so its norm-one property is also proved by hand from the C*-identity.

9.2 Gaps and workarounds

Bilinear trace positivity. Mathlib proves $0 \le \mathrm{Tr}(A)$ for a single positive-semidefinite matrix and covers sandwich forms BAB^*. The bilinear statement $0 \le \mathrm{Tr}(AB)$ for two positive-semidefinite factors is absent. For Born weights the gap costs nothing, because $\mathrm{Tr}(\rho P) = \mathrm{Tr}(P\rho P)$ for idempotent P. For `expectation_nonneg` the natural route, the C*-order equivalence between nonnegativity and Gram factorization, fails to elaborate: two continuous-functional-calculus instances are not synthesized for the matrix algebra under its product topology. The proof takes the elementary route instead, a spectral decomposition of the observable followed by a trace cycle and termwise nonnegativity, in about a dozen lines.

The ring identity. The square identity behind the Tsirelson bound is pure noncommutative ring algebra under side relations: four involutivity equations and four cross commutations. Lean has no noncommutative analogue of `linear_combination`, and `noncomm_ring` does not use hypotheses. The checked proof builds a small confluent rewrite system. Each commutation hypothesis is recast as a universally quantified, association-compatible form $b(ax) = a(bx)$, and each involution as $a(ax) = x$; applied through `simp only`, these rules move every b past every a in right-associated words and cancel adjacent equal letters. Every rewrite strictly decreases the number of letter pairs out of normal order, so the system terminates, and `abel` closes the remaining additive goal.

Higher-order motives. Two steps needed their motives spelled out. The binary span-induction principle behind the closure package of Proposition 2 does not infer its motive from the goal; the predicate is passed explicitly. Rewriting with the unit-trace equation inside a hypothesis whose proof term mentions the bundled state predicate fails the motive typecheck, because the state predicate itself contains the trace being abstracted; a term-level chain through injectivity of the real embedding replaces the rewrite.

Rewriting discipline. Unfolding the definition of the Born weight rewrites the first syntactic occurrence, which in normalized expressions is often the normalizing scalar $\mu_\rho(P)^{-1}$ instead of the intended trace argument. The affected proofs isolate each trace identity as a standalone step with explicit arguments to the trace commutation and cycling lemmas.

9.3 Affordances

The `Matrix.PosSemidef` API is complete enough that the development never unfolds the definition of positive semidefiniteness: sandwich closure, sums, scalar multiples, diagonal nonnegativity, trace nonnegativity, and the Gram construction cover every need, and the scalar lemma accepts a complex scalar that is nonnegative in the complex order, which makes the Lüders normalization a single application. Trace faithfulness is available in two forms: the conjugate-transpose criterion powers the uniqueness theorems of both expectations, and the zero-trace criterion together with the Cauchy–Schwarz property of positive matrices powers the support theorem of Section 4. On the analytic side, the C*-identity, the norm-one property of the unit, and submultiplicativity reduce the norm half of the Tsirelson bound to about forty lines over the four-typeclass signature, and the matrix case is a two-line instantiation behind the norm scope. The state-expectation bound of Section 8.1 rests on the scoped norm being definitionally the Euclidean operator norm: the type synonym is identity-transparent, so a `show` converts Euclidean-space statements to raw matrix-vector algebra, and the basis-vector, submultiplicativity, and eigenvalue-trace lemmas finish the proof in about sixty lines. Mathlib's CHSH file supplied the `IsCHSHTuple` bundle, so interoperability with the order-form theorem cost one conversion theorem.

10 Artifact evaluation and proof audit

The source is a Lake project. Its toolchain pins `leanprover/lean4:v4.29.1`; its manifest pins Mathlib commit `5e932f97dd25535344f80f9dd8da3aab83df0fe6`. Build the library with

```
lake build EventAlgebra.
```

The canonical Lean modules are publicly available in the Observer Patch Holography repository [15] at commit `49ee36e0`. The manuscript source accompanies the submission. It includes a standalone archive [14] with a byte-identical copy of those modules.

Table 2 describes the scale of the development. The counts are descriptive; the comparison in Section 11 concerns interfaces, not counts.

Module	Lines	Audited declarations
`Basic`	302	25
`Lueders`	294	14
`PartitionPinching`	474	27
`PartitionAverage`	513	30
`StateExpectation`	105	4
`Tsirelson`	275	8
`ExpectationBound`	174	4
`EventAlgebra` (root)	41	0
Total	2178	112

Table 2: Source lines and per-module counts of declarations covered by the generated axiom audit.

Every public result discussed in the paper appears under its Lean name. The modules end with `#print axioms` commands for the principal declarations, and the artifact stores the generated output. Of the 112 audited declarations, 110 report exactly propositional extensionality, classical choice, and quotient soundness, the standard Lean/Mathlib base. The CHSH ring identity and the event-commutation adapter report propositional extensionality and quotient soundness alone; the bare ring algebra needs no classical choice. No declaration reports `sorryAx`, and a source scan finds no `sorry` placeholder in the submitted modules.

The submitted API uses every orthogonality hypothesis in its orthogonal-sum theorem. The sequential-update theorem has no unused event premise. State update is typed and excludes dimension zero. The fixed-point equivalence has no nonzero-weight guard, and the averaging module carries no nonzero-block hypotheses. The commutant, span, and both expectations are bundled, and the CHSH adapters are clients of the event definitions.

The release archive contains the toolchain, Lake manifest, source modules, build command, checksums, captured axiom output, and a Creative Commons BY-NC-SA 4.0 license. It has no public release tag or persistent archive identifier.

11 Feature-level comparison

Table 3 compares compiled features. It does not treat a count of elementary lemmas as a research contribution.

Feature	Closest checked infrastructure	This artifact
Projective measurement	Isabelle/HOL projective-measurement developments; state/measurement interfaces in current Lean libraries	finite matrix event predicate, typed event/state subtypes, closure and trace-pairing lemmas; treated as prerequisite; no priority claim
Pinching	Physlib spectral pinching is a state-selected bundled CPTP map with fixed-point and geometric results	arbitrary supplied projective partition; bundled linear map and commutant; no CPTP or Physlib correspondence theorem
Retraction structure	operator-algebra conditional expectations and channel interfaces	exact range/fixed points, positive/unital/trace-preserving, bimodule, Hilbert–Schmidt geometry, map-level trace-duality uniqueness
Commutative layer	conditional expectations onto commutative subalgebras in operator-algebra practice	bundled averaging expectation onto the partition span, exact range and uniqueness, tower laws with pinching, Born-statistics preservation, classical-conditioning collapse
Lüders update	Isabelle projective measurement and general channel infrastructure	raw formula plus typed partial state API, support theorem, unguarded state fixed-point equivalence, typed naturality with arbitrary-partition pinching
CHSH/ Tsirelson	Mathlib order theorem; Isabelle upper bound; Lean CHSH rigidity	complementary norm theorem, event-level client, and state-expectation corollary; no tightness claim
Reproducibility	public CI and archived releases are common best practice	pinned Lean/Mathlib, clean artifact build instructions, checksums and static axiom audit; no DOI claim

Table 3: Direct comparison with the closest formal infrastructure.

Physlib provides more channel structure: its spectral pinching is a `CPTPMap` [16]. This artifact accepts a supplied partition that need not come from the spectrum of a state. The two APIs therefore have different inputs, and no correspondence theorem connects them.

Lean-Quantum and Lean-QIT cover a larger part of quantum information [11, 23]. Here, the scope is limited to connections among projections, commutants, spans, the two expectations, selective update, and Mathlib's CHSH interface.

12 Limitations and conclusion

The library connects projection events, Born weights, a typed Lüders update, arbitrary projective partitions, the two expectations of a partition, and event-level CHSH bounds in norm and state form. The pinching has the bundled commutant as its exact range and the average has the bundled span; each map is the unique trace-dual projection onto its range, the two compose as a tower, and both interact with Lüders conditioning by checked theorems. Each statement in the paper points to a declaration in the submitted source.

The construction follows one pattern: state algebraic content as predicates over raw matrices, move to subtypes exactly where an API promises a state, bundle the carrier and the projection

so that range and uniqueness theorems are exact, and hand finished objects to Mathlib interfaces through small adapters. The pattern is independent of measurement theory and applies to other finite-dimensional operator-algebra developments over Mathlib.

Complete positivity is not established, so neither expectation is a quantum channel in the library. There is no theorem connecting the supplied partition to Physlib's spectral pinching and no rank-one classical specialization. The CHSH bounds have no sharpness witness. The entire development is finite-dimensional and says nothing about normal maps on infinite-dimensional von Neumann algebras.

Open work consists of four items: prove complete positivity, bundle the maps as CPTP channels, connect spectral partitions to Physlib, and add a two-qubit sharpness witness for the CHSH bounds. The present artifact provides the arbitrary-partition pinching with its exact range and bimodule law, the averaging expectation with its tower and classical-conditioning laws, typed Lüders naturality, and the state-level CHSH corollary.

Declarations

Funding. No funding was received for this work.

Competing interests. The author declares no competing interests.

Author contributions. Bernhard Mueller is the sole author, designed the study, reviewed the formal development, and takes responsibility for the manuscript and artifact.

Data and code availability. No empirical data were generated. The canonical Lean source is available in the Observer Patch Holography repository [15] at commit `49ee36e0`. The Lean source, pinned dependency files, build instructions, axiom report, and checksums also accompany the submission [14], under a Creative Commons BY-NC-SA 4.0 license. No persistent archive identifier is claimed.

References

[1] Rajendra Bhatia. Pinching, trimming, truncating, and averaging of matrices. *The American Mathematical Monthly*, 107(7):602–608, 2000.

[2] Anthony Bordg, Hanna Lachnitt, and Yijun He. Certified quantum computation in Isabelle/HOL. *Journal of Automated Reasoning*, 65:691–709, 2021. https://doi.org/10.1007/s10817-020-09584-7.

[3] Paul Busch and Pekka Lahti. Lüders rule. In Daniel Greenberger, Klaus Hentschel, and Friedel Weinert, editors, *Compendium of Quantum Physics*. Springer, Berlin, Heidelberg, 2009.

[4] Boris S. Cirel'son. Quantum generalizations of Bell's inequality. *Letters in Mathematical Physics*, 4(2):93–100, 1980. https://doi.org/10.1007/BF00417500.

[5] John F. Clauser, Michael A. Horne, Abner Shimony, and Richard A. Holt. Proposed experiment to test local hidden-variable theories. *Physical Review Letters*, 23(15):880–884, 1969. https://doi.org/10.1103/PhysRevLett.23.880.

[6] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. https://doi.org/10.1007/978-3-030-79876-5_37.

[7] Mnacho Echenim. Quantum projective measurements and the CHSH inequality. *Archive of Formal Proofs*, 2021. Formal proof development, ISSN 2150-914x.

[8] Mnacho Echenim and Mehdi Mhalla. A formalization of the CHSH inequality and Tsirelson's upper-bound in Isabelle/HOL. *Journal of Automated Reasoning*, 68:Article 2, 2024. https://doi.org/10.1007/s10817-023-09689-9.

[9] Mnacho Echenim, Mehdi Mhalla, and Coraline Mori. The CHSH inequality: Tsirelson's upper-bound and other results. *Archive of Formal Proofs*, 2023. Formal proof development, ISSN 2150-914x.

[10] Masahito Hayashi. *Quantum Information Theory: Mathematical Foundation*. Graduate Texts in Physics. Springer, Berlin, Heidelberg, 2nd edition, 2017. https://doi.org/10.1007/978-3-662-49725-8.

[11] Kazumi Kasaura, Kei Tsukamoto, Kento Mori, Risa Mizuno, Takahiro Namatame, Yuta Oriike, Masaya Taniguchi, Sho Sonoda, and Hayata Yamasaki. Lean-Quantum: Toward AI-assisted formalization of quantum information, 2026. arXiv:2607.05492, https://doi.org/10.48550/arXiv.2607.05492.

[12] Gerhart Lüders. Über die zustandsänderung durch den meßprozeß. *Annalen der Physik*, 8: 322–328, 1951. English translation by K. A. Kirkpatrick, *Concerning the state-change due to the measurement process*, Annalen der Physik **15**(9), 663–670 (2006).

[13] Kim Morrison and Mathlib contributors. Mathlib.algebra.star.chsh. Mathlib documentation, 2026. URL https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/Star/CHSH.html. Accessed 20 July 2026.

[14] Bernhard Mueller. EventAlgebra: A Lean 4 formalization of finite quantum event algebras, 2026. Software artifact accompanying this article; built with toolchain `leanprover/lean4:v4.29.1` over Mathlib.

[15] Observer Patch Holography contributors. Observer Patch Holography: Source repository. https://github.com/FloatingPragma/observer-patch-holography, 2026. Commit 49ee36e01eb245d4d508e82c9b08fd47f7a85243; accessed 21 July 2026.

[16] Physlib contributors. Quantuminfo.finite.pinching: Pinching channels. Lean library documentation, 2026. URL https://ohaithe.re/Lean-QuantumInfo/QuantumInfo/Finite/Pinching.html. Accessed 20 July 2026.

[17] Physlib/QuantumInfo contributors. QuantumInfo: A Lean 4 library for quantum information, 2026. Software project; finite mixed states as positive-semidefinite trace-one matrices, POVMs, and Born-rule measurement.

[18] Masamichi Takesaki. Conditional expectations in von Neumann algebras. *Journal of Functional Analysis*, 9(3):306–321, 1972. https://doi.org/10.1016/0022-1236(72)90004-3.

[19] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. https://doi.org/10.1145/3372885.3373824.

[20] Hisaharu Umegaki. Conditional expectation in an operator algebra. *Tôhoku Mathematical Journal*, 6(2–3):177–181, 1954. https://doi.org/10.2748/tmj/1178245177.

[21] John von Neumann. *Mathematische Grundlagen der Quantenmechanik*. Springer, Berlin, 1932. English translation: *Mathematical Foundations of Quantum Mechanics*, Princeton University Press, Princeton, 1955.

[22] Tianrun Zhao and Nengkun Yu. Formalizing CHSH rigidity in Lean 4, 2026. arXiv:2604.03884, https://doi.org/10.48550/arXiv.2604.03884.

[23] Chengkai Zhu, Ziao Tang, Guocheng Zhen, Yimeng Cao, Yusheng Zhao, Ranyiliu Chen, Xuanqiang Zhao, Lei Zhang, and Xin Wang. Lean-QIT: Towards a formal infrastructure for quantum information theory, 2026. arXiv:2607.09632, https://doi.org/10.48550/arXiv.2607.09632.